



TransOptAS: Transformer-Based Algorithm Selection for Single-Objective Optimization

Gjorgjina Cenikj
Computer Systems Department
Jožef Stefan Institute
Jožef Stefan International
Postgraduate School
Ljubljana, Slovenia
gjorgjina.cenikj@ijs.si

Gašper Petelin
Computer Systems Department
Jožef Stefan Institute
Jožef Stefan International
Postgraduate School
Ljubljana, Slovenia
gasper.petelin@ijs.si

Tome Eftimov
Computer Systems Department
Jožef Stefan Institute
Ljubljana, Slovenia
tome.eftimov@ijs.si

ABSTRACT

Driven by the success of transformer models in representation learning across different Machine Learning fields, this study explores the development of a transformer-based Algorithm Selection (AS) model. **We demonstrate the training of a transformer architecture for AS on a collection of single-objective problem instances generated through a random function generator. The transformer model predicts a numeric performance indicator for all of the algorithms in a given algorithm portfolio, relying solely on raw samples from the objective functions. The transformer model consistently outperforms a single best solver, indicating that AS can be performed without the need for intermediate feature construction.** A comparison to the classic Exploratory Landscape Analysis (ELA) approach shows that the transformer model provides comparable results, with the ELA model commonly outperforming the transformer model with differences around 0.02 in terms of the loss metric.

CCS CONCEPTS

• Computing methodologies → Machine learning; Learning latent representations; Supervised learning; • Theory of computation → Design and analysis of algorithms.

KEYWORDS

algorithm selection, single-objective continuous optimization, representation learning, problem landscape features

ACM Reference Format:

Gjorgjina Cenikj, Gašper Petelin, and Tome Eftimov. 2024. TransOptAS: Transformer-Based Algorithm Selection for Single-Objective Optimization. In *Genetic and Evolutionary Computation Conference (GECCO '24 Companion)*, July 14–18, 2024, Melbourne, VIC, Australia. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3638530.3654191>

1 INTRODUCTION

The process of selecting the most suitable optimization algorithm for a given optimization problem, known as Algorithm Selection

(AS), is motivated by the potential to leverage the diverse performance exhibited by different algorithms across various sets of problem instances. Typically, the AS task involves considering the properties of the problem instance, which are described using a numerical vector representation often referred to as problem landscape features. Machine Learning (ML) models can utilize these features to learn the relationship between problem instances and algorithm performance, providing recommendations for the best algorithm suited to a particular problem instance.

In the realm of single-objective numerical optimization, the majority of works on AS employ Exploratory Landscape Analysis (ELA) [9] features to represent problem instances. While ELA features are valuable for characterizing optimization problems, they have also been shown to have limitations, such as sensitivity to the sample size and the sampling method employed for solution sampling from the decision space [12, 17], and lack of invariance to transformations such as scaling and shifting of the problem [17]. AS models using ELA features have also been shown not to generalize well to unseen problems [10]. These issues introduce the need for novel features that will better capture the relations between a problem instance and algorithm performance.

Recent research efforts have been directed toward the use of different deep-learning approaches for characterizing optimization problem instances, such as autoencoders [15], and fitness maps [11].

Our contribution: Motivated by the success of transformer-based models [16] in various fields and applications, **in this paper, we investigate the potential of developing a transformer-based AS model that is trained solely on raw samples of the objective function. This approach enables the model to autonomously learn representations of the problem, rather than being limited to pre-defined features. We demonstrate the training of a transformer architecture for AS on a set of single-objective problem instances which are generated using a random function generator** [14]. We show that a transformer model can successfully be trained for AS using only raw samples of the objective functions, providing comparable results to the common ELA approach, with the ELA model outperforming the transformer model with differences around 0.02.

Outline: The remainder of the paper is organized as follows. In Section 2, we explain the data generation, model architecture, and the comparison to the ELA-based model. Section 3 contains the results of the transformer-based AS and a comparison to an ELA-based model. Section 4 concludes the paper.

Reproducibility: The code and data used in the experiments are available at <https://github.com/gjorgjinac/TransOptAS>.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
GECCO '24 Companion, July 14–18, 2024, Melbourne, VIC, Australia
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0495-6/24/07.
<https://doi.org/10.1145/3638530.3654191>

2 METHODOLOGY

In this section, we first present the problem portfolio, i.e., the set of problem instances used for training the AS model and the sampling process. We then present the algorithm portfolios and the algorithm performance metric. We explain the model architecture, the AS loss and the model training. Finally, we outline the comparative analysis between the proposed methodology and the conventional ELA-based AS approach.

2.1 Problem Portfolio

We generate 5,000 random problem instances of dimension 3 and 10, using the random function generator proposed in [14]. The generator constructs a tree representation by randomly combining mathematical operands and operators from a predefined pool, where each operand and operator has a specific probability of being selected. We use the Python implementation of the random function generator, which was originally implemented in Matlab and was re-implemented in Python in [7].

One limitation of the generator is that it can produce functions which return constant values for all inputs. Since it does not make sense to optimize such functions, we remove them from the initial set of created functions. The generator can also produce duplicate functions or functions with substantial similarities. Including the same (or nearly identical) problem instances in both the training set and the test set can lead to an unfair evaluation. For this reason, we use the SELECTOR methodology [2] to remove duplicate problem instances from the data. This is accomplished in the following way. We first generate a fixed sample of candidate solutions (x-values) with a sample size of $50d$ where d is the problem dimensionality. We then evaluate all of the generated functions in this sample to obtain the objective function values (y-values).

The objective function values are scaled in the range $[0,1]$ within the samples of each problem instance individually using min-max scaling. Next, we calculate the Euclidean distance between the objective function values of each pair of problem instances, obtained on an identical sample of candidate solutions. The SELECTOR methodology then involves the creation of a graph where nodes represent problem instances, and each pair of nodes is connected with an edge if the Euclidean distance of their objective function values is less than 0.01. The Maximal Independent Set algorithm [13] is then executed on the graph, producing a set of problem instances where no duplicates are present. This procedure resulted in 2,638 $3d$ functions and 2,696 $10d$ functions used for the model training.

Please note that the use of SELECTOR is not a required step in our proposed methodology, but it is merely a way to mitigate the issue of duplicate problem instances present in the data.

2.2 Sampling

The first step of the proposed methodology involves the generation of samples from which the problem representations are calculated. The samples are obtained using Latin Hypercube Sampling [8] with a sample size of $50d$, where d is the problem dimensionality. We choose this sample size since it has been shown to work well both for the ELA features [5], and for training a transformer-based model for problem classification [3]. The objective values of the samples (y-values) are scaled to be in the range $[0,1]$ using min-max scaling,

while the candidate solutions (x-values) are kept in their original range.

2.3 Algorithm Portfolio

The algorithm portfolio includes 12 distinct configurations of the Particle Swarm Optimization (PSO) [4] algorithm, with specific parameter settings detailed in our github repository.

The execution of these algorithms is facilitated through the *py-moo* [1] Python library on all designated problem instances. The algorithms operate within a fixed-budget scenario, with algorithm performance metrics being recorded at budgets of 10, 30, and 50 iterations. For a $3d$ problem, a 50-iteration budget translates to a total of 1,500 function evaluations (50 iterations with a population size of 30). All algorithms use Latin Hypercube Sampling [8] to construct the initial population, and have a population size of $10d$. We perform ten executions of each algorithm on all problem instances, and capture the overall algorithm performance on each problem instance as described in the following subsection.

2.4 Algorithm Performance Metric

We adopt a fixed-budget scenario for assessing the algorithms, meaning that we record the best objective function value obtained by each algorithm after a certain number of iterations. Since the random problem instances do not have a known optimum, we cannot use the conventional approach of estimating the achieved precision (distance of the best-found solution to the optimum). Instead, we propose a different score, defined as follows.

Let y_{arp} be the best solution (the lowest objective function value) found by algorithm A in run r for problem instance p . We scale y_{arp} by the range of the best solutions found by the other algorithms for the same problem instance and the same initial population (same run). Let b_{rp} and w_{rp} be the best and worst solution found for problem instance p in run r by any of the algorithms under comparison. The scaled best objective function value achieved by algorithm A in run r on problem instance p is given by:

$$s_{arp} = \frac{y_{arp} - b_{rp}}{w_{rp} - b_{rp}} \quad (1)$$

This score captures how much the best solution found by algorithm A in run r was better than the other algorithms executed with the same initial population. We calculate the final algorithm score s_{ap} , capturing the performance of algorithm A on problem instance p across all algorithm runs, by simply taking the median of all s_{arp} .

2.5 Model Architecture

The heart of the original transformer architecture is its encoder-decoder structure. In our case, we only use the encoder part of the transformer architecture, which consists of a multi-head self-attention module and a position-wise feedforward network.

Since transformer models have become widely used across different domains and our implementation of the encoder is essentially identical to the original, we omit an exhaustive background description of the transformer architecture and refer readers to the original work [16]. One thing to note is that the original transformer architecture is invariant to the ordering of the input data. In other ML fields such as natural language processing or time

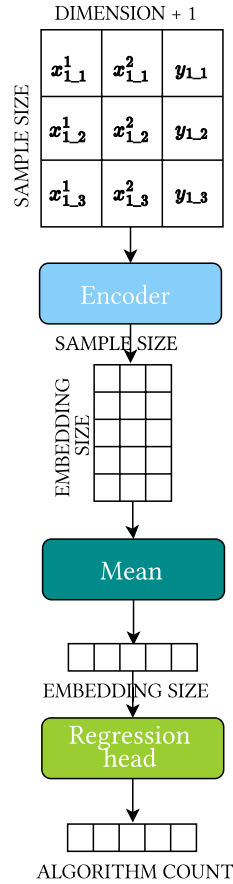


Figure 1: Model architecture with example inputs being samples of two-dimensional optimization problems

series analysis, where the ordering of the input data (i.e. the order of words in a sentence or the time component of a time series) is important, a positional encoding is added to the input embeddings to model the sequence information. In our case, since the samples of an optimization problem do not have an inherent ordering, the architecture should be kept order-invariant, so we do not add a positional encoding.

Figure 1 features the full model architecture and an example of a single training sample. The input to the model are the samples from the optimization problems. A single training instance of the ML model is a matrix of shape $[s, d + 1]$, where s is the number of samples from each problem instance, and d is the dimensionality of the problem. In this case, the second dimension of the input matrix is $d + 1$, since the value of the objective function is also included.

The encoder produces an embedding of size e , which is a specified model parameter, for each of the given samples from the optimization problem, i.e. it outputs a matrix of the shape $[s, e]$. In order to obtain a single, flat representation of the problem, we take the mean of the obtained matrix. The produced problem representation is then fed into a regression head, producing a score in the range $[0, 1]$ for each algorithm in the algorithm portfolio.

2.6 Algorithm Selection Loss

To capture the true performance of the algorithm predicted to be the best by the algorithm selector, we calculate the loss of the AS:

$$loss = \frac{1}{|\mathcal{P}|} \sum_{p \in \mathcal{P}} 1 - (s_{sp} - s_{bp}) \quad (2)$$

where s_{sp} is the true score of the selected algorithm which was predicted to have the best performance for problem instance p , while s_{bp} is the score of the best-performing algorithm on problem instance p (i.e. the virtual best solver). This score would get a value of zero if the worst algorithm is predicted to be the best for all problem instances, and a value of one if the best algorithm is predicted to be the best for all problem instances. Please note that the metric is supposed to be maximized, so higher values are better.

2.7 Model Training

The model is trained to minimize the sum of the AS loss as defined in Section 2.6 and the sum of the squared errors between the predicted and true scores of the algorithms. The reasoning behind such a loss function is to encourage the model to emphasize the importance of having a good first-ranked algorithm, while still taking into consideration the scores of the other algorithms.

The Adam optimizer is used with a maximal learning rate of 0.001. The training is executed for a maximum of 200 epochs, with an early stopping mechanism that prevents overfitting by terminating the training process if the validation loss does not observe a decrease of at least 0.001 for five epochs. The model training is repeated 10 times with different train-validation-test splits. Random splitting is used to split the instances into training, validation, and test data with a ratio 80-10-10. The training data is used to train the model, the validation data is used to determine when to perform the early stopping, while the test data is used to evaluate the model. We train a separate transformer model for each problem dimension and each algorithm execution budget. The parameter values used for training the transformer architecture can be found in our github repository.

2.8 Comparison to the ELA approach

The proposed representation learning method is compared to the performance of the most commonly used ELA features. For this purpose, we adopt the conventional methodology of using ELA features and a Random Forest model trained for AS.

To ensure a fair comparison, ELA features are computed on the same set of 50d samples that are used to train the transformer model. We compute ELA features from the following categories using the *flacco* R library [6]: *basic*, *disp*, *ela_distr*, *ela_level*, *ela_meta*, *ic*, *nbc*, and *pca*. In this way, we calculate a total of 93 features, on which the model is trained to predict the problem class. Some of these features contain missing values, which we remove before training the AS model. The model training is repeated 10 times in order to ensure the robustness of the results, where the splitting of problem instances into folds is consistent with the splitting used for the transformer model.

3 RESULTS

Figure 2 shows the results of the AS, with the loss presented on the vertical axis, and the algorithm execution budget on the horizontal

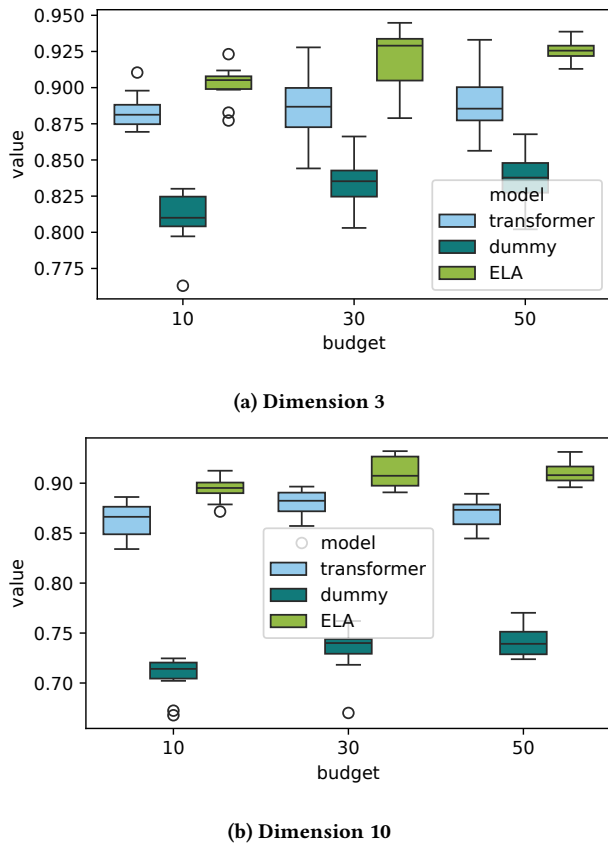


Figure 2: Losses achieved by the dummy model, the transformer-based model and the ELA-based model

axis. The differently colored barplots indicate different models. We compare the transformer model to the RF model using ELA features. We also include a simple dummy baseline, predicting the mean score of each algorithm on the training set.

We can see that both the ELA and the transformer features outperform the dummy in both problem dimensions. The ELA features provide better results than the transformer features, however, the difference between them does not seem substantial. The difference between the losses achieved by the ELA model and the transformer model, aggregated across all budgets has a median of 0.02.

4 CONCLUSION

In this paper, we propose a novel approach to algorithm selection (AS) for single objective optimization problem instances, involving the training of a transformer architecture using only raw problem samples. We show that a transformer model can be successfully trained on a benchmark of randomly generated problem instances.

Overall, the results demonstrate the potential of our proposed approach. Even though the ELA model provides slightly better results compared to the transformer, we have to note that this is only a preliminary study of the opportunity of training transformer models for AS, while substantially more effort has been invested into the development of the ELA features over the past two decades.

5 ACKNOWLEDGEMENTS

Funding in direct support of this work: Slovenian Research Agency: research core funding No. P2-0098, young researcher grants No. PR-12393 to GC and No. PR-11263 to GP, and project No. J2-4460.

REFERENCES

- [1] J. Blank and K. Deb. 2020. Pymoo: Multi-Objective Optimization in Python. *IEEE Access* 8 (2020), 89497–89509.
- [2] Gjorgjina Cenikj, Ryan Dieter Lang, Andries Petrus Engelbrecht, Carola Doerr, Peter Korošec, and Tome Eftimov. 2022. SELECTOR: selecting a representative benchmark suite for reproducible statistical comparison. In *Proceedings of the Genetic and Evolutionary Computation Conference (Boston, Massachusetts) (GECCO '22)*. Association for Computing Machinery, New York, NY, USA, 620–629. <https://doi.org/10.1145/3512290.3528809>
- [3] Gjorgjina Cenikj, Gašper Petelin, and Tome Eftimov. 2023. TransOpt: Transformer-based Representation Learning for Optimization Problem Classification. *2023 IEEE Symposium Series on Computational Intelligence (2023)*. <http://arxiv.org/abs/2311.18035>
- [4] J. Kennedy and R. Eberhart. 1995. Particle swarm optimization. In *Proceedings of ICNN'95 - International Conference on Neural Networks*, Vol. 4. 1942–1948 vol.4. <https://doi.org/10.1109/ICNN.1995.488968>
- [5] Pascal Kerschke, Mike Preuss, Simon Wessing, and Heike Trautmann. 2016. Low-Budget Exploratory Landscape Analysis on Multiple Peaks Models. 229–236. <https://doi.org/10.1145/2908812.2908845>
- [6] Pascal Kerschke and Heike Trautmann. 2019. *Comprehensive Feature-Based Landscape Analysis of Continuous and Constrained Optimization Problems Using the R-Package Flacco*. Springer International Publishing, Cham, 93–123. https://doi.org/10.1007/978-3-030-25147-5_7
- [7] Fu Xing Long, Bas van Stein, Moritz Frenzel, Peter Krause, Markus Gitterle, and Thomas Bäck. 2022. Learning the Characteristics of Engineering Optimization Problems with Applications in Automotive Crash. In *Proceedings of the Genetic and Evolutionary Computation Conference (Boston, Massachusetts) (GECCO '22)*. Association for Computing Machinery, New York, NY, USA, 1227–1236. <https://doi.org/10.1145/3512290.3528712>
- [8] Jaroslav Mencík. 2016. Latin Hypercube Sampling. In *Concise Reliability for Engineers*, Jaroslav Mencik (Ed.), IntechOpen, Rijeka, Chapter 16. <https://doi.org/10.5772/62370>
- [9] Olaf Mersmann, Bernd Bischl, Heike Trautmann, Mike Preuss, Claus Weihs, and Günter Rudolph. 2011. Exploratory landscape analysis. In *Proceedings of the 13th annual conference on Genetic and evolutionary computation*. 829–836.
- [10] Gašper Petelin and Gjorgjina Cenikj. 2023. How Far Out of Distribution Can We Go With ELA Features and Still Be Able to Rank Algorithms?. In *2023 IEEE Symposium Series on Computational Intelligence (SSCI)*. 341–346. <https://doi.org/10.1109/SSCI52147.2023.10371880>
- [11] Raphael Patrick Prager, Moritz Vinzent Seiler, Heike Trautmann, and Pascal Kerschke. 2021. Towards Feature-Free Automated Algorithm Selection for Single-Objective Continuous Black-Box Optimization. In *2021 IEEE Symposium Series on Computational Intelligence (SSCI)*. 1–8. <https://doi.org/10.1109/SSCI50451.2021.9660174>
- [12] Quentin Renau, Carola Doerr, Johann Dreö, and Benjamin Doerr. 2020. *Exploratory Landscape Analysis is Strongly Sensitive to the Sampling Strategy*. 139–153. https://doi.org/10.1007/978-3-030-58115-2_10
- [13] Robert Endre Tarjan and Anthony E Trojanowski. 1977. Finding a maximum independent set. *SIAM J. Comput.* 6, 3 (1977), 537–546.
- [14] Ye Tian, Shichen Peng, Xingyi Zhang, Tobias Rodemann, Kay Chen Tan, and Yaochu Jin. 2020. A Recommender System for Metaheuristic Algorithms for Continuous Optimization Based on Deep Recurrent Neural Networks. *IEEE Transactions on Artificial Intelligence* 1, 1 (2020), 5–18. <https://doi.org/10.1109/TAI.2020.3022339>
- [15] Niki van Stein, Fu Xing Long, Moritz Frenzel, Peter Krause, Markus Gitterle, and Thomas Bäck. 2023. DoE2Vec: Deep-learning Based Features for Exploratory Landscape Analysis. (03 2023).
- [16] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. <https://doi.org/10.48550/ARXIV.1706.03762>
- [17] Urban Škvorc, Tome Eftimov, and Peter Korošec. 2021. The Effect of Sampling Methods on the Invariance to Function Transformations When Using Exploratory Landscape Analysis. In *2021 IEEE Congress on Evolutionary Computation (CEC)*. 1139–1146. <https://doi.org/10.1109/CEC45853.2021.9504739>